Response to Gyan (Gnana Rajnan) — Pipeline DB & Tables Specification

LocalAI TV — AI Pipeline Database Design · Endorsed into Plan v1.3

To: Gyan / Gnana Rajnan (AI Pipeline Engineer) **From:** LocalAI TV Architecture Team **Subject:** Endorsement of your AI Pipeline DB design + the two-database federation model + integration notes **Date:** 15 May 2026 **Status:** ✅ Your design ADOPTED into Plan v1.3 with integration clarifications

---

Dear Gyan,

Your **Pipeline DB & Tables Specification** is the highest-quality technical artifact this project has produced. You did a hands-on codebase walkthrough and produced a production-grade, scalable database design with disciplined phasing. It is endorsed almost entirely as-is.

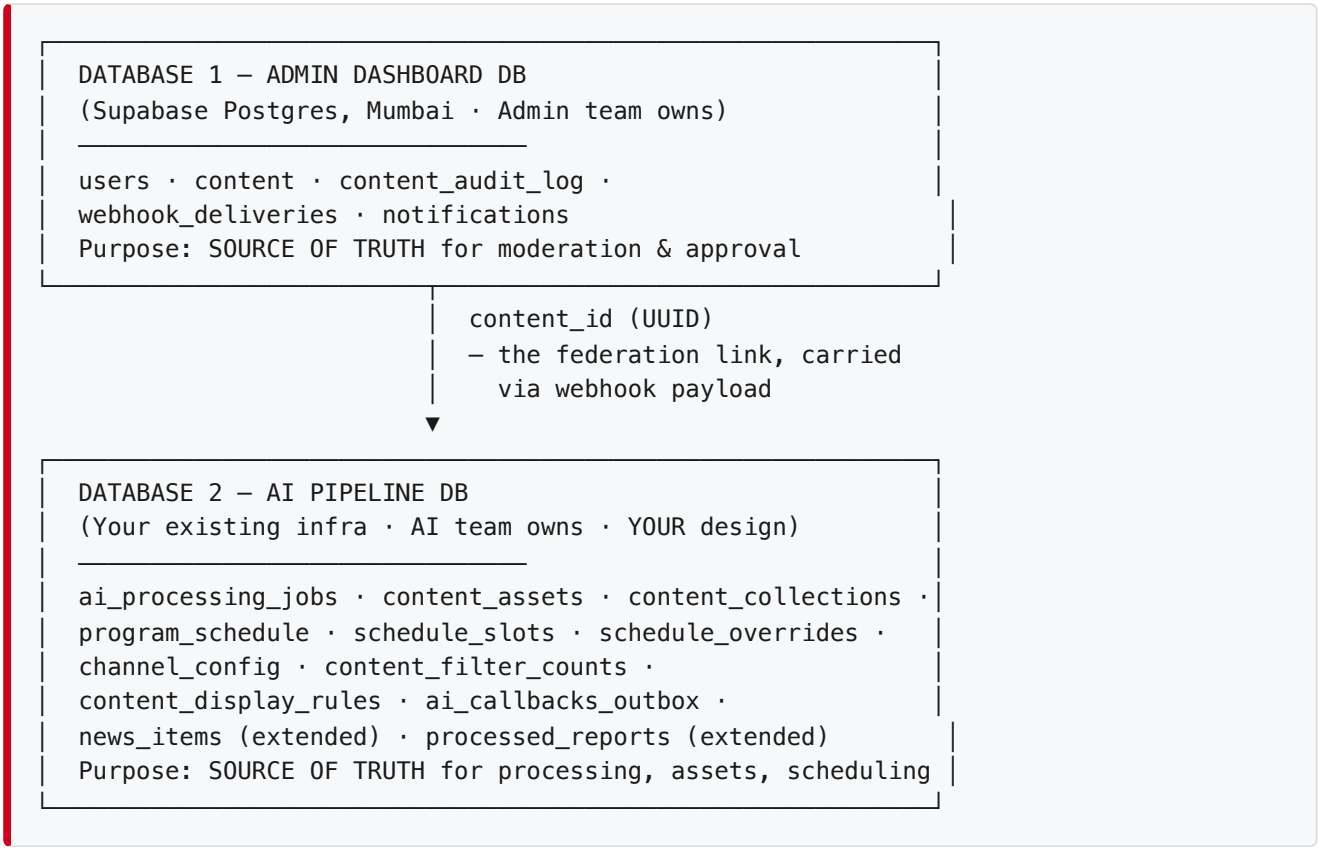
This letter:

1. Confirms the **two-database federation model** your work reveals
2. Endorses each of your proposed tables with integration notes
3. Clarifies the one overlap point (so there's no duplication confusion)
4. Confirms what goes into Plan v1.3

---

1. The Key Architectural Insight Your Document Revealed

Your spec made it clear that **LocalAI TV has TWO databases, not one** — and that is the correct design.



Why two databases is the right call (confirming your implicit design):

Reason	Detail
Separation of concerns	Admin team owns moderation DB; AI team (you) owns processing DB
Different scale profiles	Admin DB = low volume (decisions); AI DB = very high volume (600K jobs/day, millions of assets at peak)
Failure isolation	If Admin DB briefly down, your pipeline keeps processing queued work
Team autonomy	You evolve your schema without coordinating every change with Admin team
You already have it	`processed_reports`, `news_items`, `app_state`, `item_events`, `bulletin_events`, `incidents` exist — we extend, not replace

The single link is `content_id` (UUID generated by Admin Dashboard) flowing through the webhook → stored in your `ai_processing_jobs.content_id` → referenced in your callback → Admin updates `content.status`. Clean federated architecture. **Endorsed.**

---

2. Endorsement of Each Proposed Table

Your proposal	Verdict	Integration note
§2.1 `channel_config`, `content_filter_counts` (precomputed, not live queries)	✅ **Adopt**	Directly solves the 3,000-channel filter problem AND Abishek's analytics-overload concern (#7). Precomputed counts = no slow live filtering.
§2.2 `program_schedule`, `schedule_slots`, `schedule_overrides`, `channel_program_rules`	✅ **Adopt — replaces Admin's simpler design**	Your scheduler schema is better than my `publishing_queue`. v1.3 adopts YOUR schema. Schedule rules move from `celery_app.py` hardcoding into these DB tables.
§2.3 `news_items` + `content_id`, `ai_job_id`, `channel_id`, `source_type`	✅ **Critical — this IS the federation link**	This is exactly how the two databases connect. Endorsed.
§2.4 `content_collections` + `content_assets`	✅ **Brilliant**	Decouples scheduler from physical S3 paths. Mandatory for 3,000-channel scale. The scheduler calls content by collection, never by physical folder. Adopted into v1.3 §8.
§2.5 `processing_mode = notebooklm_direct`, `asset_role = final_video`	✅ **Consistent**	Aligns with v1.2's `processing_mode` enum. NotebookLM content skips Gemini/TTS, tracked but not reprocessed.
§2.6 `content_templates`, `content_display_rules`, `content_rotation_state`	✅ **Adopt**	Birthday-on-date, shopping-for-a-week, auto-expiry, rotation. v1.2 §13 mentioned this conceptually; your schema makes it concrete and admin-manageable.
§2.7 `lifecycle_status`, `archive_key`, `content_retention_rules`	✅ **Complements S3 lifecycle**	DB tracks logical state (active/archived/expired); S3 lifecycle policy handles physical tiering (Standard→IA→Glacier). Both layers, clean division.
§4.1 `ai_processing_jobs` (full lifecycle tracking)	✅ **Adopt**	The canonical per-content job record on the AI side. See overlap note below.
§4.2 `content_assets` (every S3 file: input/intermediate/output)	✅ **Adopt**	"S3 files must not be hidden in JSON blobs" — exactly right. Replaces reliance on Admin's `media_urls` JSONB for pipeline-side asset tracking.
§4.7 `ai_callbacks_outbox` (transactional outbox)	✅ **Production best-practice**	This is the textbook **transactional outbox pattern**. Directly addresses Abishek's webhook-reliability concern (#1). Excellent. Adopted into v1.3 §15.
§5 Minimal first-phase list (10 tables, "don't build all at once")	✅ **Disciplined and correct**	Endorsed exactly. Phase the table creation.
§6 Changes to existing tables (`news_items`, `processed_reports`, `item_events`, `bulletin_events`)	✅ **Adopt**	Additive columns only — no destructive changes. Safe.

Your proposal	Verdict	Integration note
§7 Worker/DB interaction model (media/build/maintenance/beat)	✅ **Clear and correct**	Each worker's DB responsibility is well-defined. Endorsed.
§8 Final recommendations	✅ **All textbook-correct**	"RabbitMQ for work movement only · Redis for cache/locks only · S3 is media store · DB is media index" — keep all of these as stated principles in v1.3.

---

3. The One Overlap Point — Clarified (no duplication)

Your `ai_processing_jobs` table superficially overlaps with two Admin-side tables. They are **complementary, on different sides, and all three are needed**:

Table	Side	Tracks	Owner
`webhook_deliveries`	Admin DB	Did the Admin→AI webhook POST succeed? (delivery attempt)	Admin team
`ai_processing_jobs`	**AI DB**	**Job lifecycle inside your pipeline (queued→processing→done)**	**You**
`ai_callbacks_outbox`	AI DB	Did the AI→Admin callback POST succeed? (delivery attempt)	You

Together these give **end-to-end traceability**:

- Admin knows it sent the webhook (`webhook_deliveries`)
- You know the job's processing state (`ai_processing_jobs`)
- You know whether you successfully notified Admin (`ai_callbacks_outbox`)

No conflict. No duplication. Build all three. The `content_id` ties them together across both databases.

---

4. Minor Integration Notes for Your Build

1. **`content_id` is a UUID v4** generated by Admin Dashboard. Globally unique across all states/districts/constituencies/categories. Safe to use as a partition key or filename prefix without geographic suffixes.
2. **`approved_version`** — for v1 always `1` (re-approval deferred to a later phase). Your `idempotency_key = content_id + "_v" + approved_version` format is adopted exactly. Future-proofed.
3. **`content_assets.object_key`** convention — recommend matching the v1.2/v1.3 S3 path structure: `ai-processed/<state>/<district>/<constituency>/<category>/<content_id>/<asset_role>.<ext>`
4. **`content_collections`** — the Admin Dashboard does NOT need to know about collections. It only sends `content_id` + `category` + geography. Your build-worker maps these to a collection via `channel_program_rules` . The collection abstraction is entirely AI-pipeline-side. Clean.

5. `ai_callbacks_outbox` **retry semantics** — align with the 4xx/5xx contract: 5xx → retry (transient), 4xx → dead-letter (permanent, don't retry). Documented in v1.3 §15.7.
6. **Schedule tables ownership** — `program_schedule` / `schedule_slots` / `schedule_overrides` live in the AI Pipeline DB but are *edited* via the Admin Dashboard UI (super admin manages broadcast slots). The Admin Dashboard will call a small AI-side API to write these, OR (simpler) the Admin UI writes directly to the AI Pipeline DB for these specific tables. We'll finalize that interface in Phase 1 — your call on which you prefer (API vs direct DB write). Note this as an open coordination item.

---

5. What Goes Into Plan v1.3

v1.3 change	Source
NEW §7b — AI Pipeline DB schema (your 10 first-phase tables, verbatim)	Your §4, §5
NEW §7c — Two-database federation model + `content_id` link	This analysis
§8 rewrite — `content_collections` logical-folder model (no physical-folder dependency)	Your §2.4
§13 rewrite — DB-driven scheduler (`program_schedule` / `schedule_slots` / `schedule_overrides`) replaces `publishing_queue`	Your §2.2
NEW §13a — `content_display_rules` (birthday/shopping/car-sale expiry & rotation)	Your §2.6
§15 update — `ai_callbacks_outbox` transactional outbox	Your §4.7
§17b update — `content_filter_counts` precomputed counts for 3,000-channel filtering	Your §2.1
§6 — `news_items` / `processed_reports` extension columns	Your §6
§20 — open coordination item: schedule-table write interface (API vs direct DB)	Note #6 above

---

6. One Open Coordination Item (needs your input)

The only thing not fully decided: **how does the Admin Dashboard write to the schedule tables** (`program_schedule` , `schedule_slots` , `schedule_overrides`) which live in YOUR database?

Two options — your preference:

- **Option A — Admin calls an AI-side API:** Admin Dashboard calls `POST https://pipeline.localaitv.com/api/schedule/...` and your FastAPI writes to your DB. Clean separation, more code.
- **Option B — Admin writes directly to AI Pipeline DB** for these 3 tables only (read-only for everything else). Less code, tighter coupling.

My recommendation: Option A (API) — preserves the two-database separation cleanly and lets you validate schedule changes server-side. But it's your database, so your call. Please advise so we can lock it in Phase 1.

---

7. Closing

Your document is what a senior data architect produces. The phasing discipline ("build only these 10 tables first, don't do the full set at once") is exactly right. The transactional outbox pattern for callbacks shows real production maturity.

Plan v1.3 adopts your design. The two-database federation is now the official architecture. The only open item is the schedule-table write interface (§6 above) — please advise.

You and Sameer have, together, produced a complete and unambiguous integration contract. The team's work is exceptional.

With respect,

LocalAI TV Architecture Team LocalAI Media Network Pvt Ltd CIN: U63910KA2025PTC212593 Hyderabad, India

---

Plan v1.3 (separate document) reflects your DB design in new §7b, §7c, and the §13 scheduler rewrite.